

Fase 1 — Análise e Mapeamento

Após a leitura integral dos cinco ficheiros ([SegInf_inv_2324_EN_v1.pdf](#), [SegInf_inv_2324_Recurso_v1.pdf](#), [SegInf_inv1920_ENormal.pdf](#), [SegInf_inv2122_ENormal.pdf](#), [SegInf_ver2122_ERecurso.pdf](#)), foi possível identificar os seguintes temas recorrentes, agrupados por afinidade lógica e pela frequência com que são avaliados.

Temas Centrais e Frequência (ordem inferida para progressão didática):

1. **Criptografia Simétrica em Bloco** (Presente em 5/5 exames): Modos de operação (ECB, CBC, GCM), Vetor de Inicialização (IV), *padding*, determinismo, dimensão da chave.
2. **Funções de Hash e MAC (Message Authentication Code)** (Presente em 4/5 exames): Propriedades das funções de hash (resistência a colisões, etc.), construção de MAC, HMAC, *salt* para *passwords*, diferença entre MAC e assinatura digital.
3. **Criptografia Assimétrica e Assinatura Digital** (Presente em 4/5 exames): Chaves pública/privada, não-repúdio, dimensão das chaves, *Perfect Forward Secrecy* (PFS).
4. **Infraestrutura de Chave Pública (PKI) e Certificados X.509** (Presente em 5/5 exames): Certificados raiz, intermédios e folha, cadeia de confiança, *TrustStore*, verificação de assinatura de certificados, campos do certificado, *Common Name* (CN), autoassinatura.
5. **Biblioteca JCA (Java Cryptography Architecture)** (Presente em 5/5 exames): Princípios de *design* (independência de algoritmo), classes *Cipher*, *Mac*, *X509Certificate*, modos de operação (*ENCRYPT_MODE*, *WRAP_MODE*), métodos *update/doFinal*.
6. **Protocolo TLS (Transport Layer Security)** (Presente em 5/5 exames): Fases do *handshake*, *client_random/server_random*, deteção de *downgrade attacks*, *Record Protocol*, derivação de chaves, autenticação de cliente e servidor, *Perfect Forward Secrecy*.
7. **Autenticação de Utilizadores e Armazenamento de Passwords** (Presente em 5/5 exames): Armazenamento inseguro (cifra reversível), uso de *salt* e funções de hash lentas, ataques de dicionário e pré-computação (*rainbow tables*), ataques à interface de autenticação.
8. **Modelos de Controlo de Acessos** (Presente em 5/5 exames): RBAC (Role-Based Access Control), conceitos de *U* (utilizadores), *R* (roles), *P* (permissões), *UA* (User Assignment), *PA* (Permission Assignment), *RH* (Role Hierarchy, $\prec$), sessões, RBAC1.
9. **Protocolos de Autorização e Autenticação Federada: OAuth 2.0 e OpenID Connect (OIDC)** (Presente em 5/5 exames): Fluxo *Authorization Code Grant*, *endpoints* (autorização, *token*, *UserInfo*), *scope*, *access_token*, *id_token*, *client_id/client_secret*, parâmetro *state* (para CSRF), ataques relacionados.

A progressão lógica inferida é a apresentada acima: das primitivas criptográficas mais básicas (simétrica) para as mais complexas (assimétrica), seguindo para a infraestrutura que as suporta (PKI) e as bibliotecas de programação que as implementam (JCA). A partir daí, abordam-se os protocolos de segurança de rede (TLS), a segurança de aplicações (autenticação local e federada com OAuth2/OIDC) e, por fim, os modelos que regem a autorização interna dos sistemas (RBAC).

Fase 2 — Produção do Material Didático

Resumo de Segurança Informática

Este documento resume os principais conceitos teóricos da disciplina de Segurança Informática, organizados de forma lógica para estudo.

1. Fundamentos de Criptografia Simétrica

A criptografia simétrica utiliza a **mesma chave secreta** k para as operações de cifra (E) e decifra (D). É adequada para garantir a **confidencialidade** de grandes volumes de dados.

1.1. Cifra em Bloco e Modos de Operação

Algoritmos como **AES (Advanced Encryption Standard)** e DES operam sobre blocos de tamanho fixo (ex: 128 bits para AES). Para cifrar mensagens de tamanho arbitrário, são necessários **modos de operação**.

Modo	Descrição e Propriedades Críticas
ECB (Electronic Code Book)	Cada bloco é cifrado independentemente com a mesma chave. Problema: É determinístico e não esconde padrões na mensagem original (blocos idênticos de texto claro produzem blocos idênticos de criptograma). Não recomendado para a maioria dos casos.
CBC (Cipher Block Chaining)	Cada bloco de texto claro sofre um XOR com o criptograma do bloco anterior antes de ser cifrado. O primeiro bloco sofre XOR com um Vetor de Inicialização (IV) aleatório e imprevisível. O IV não precisa de ser secreto , mas deve ser conhecido para a decifra. Torna a cifra não-determinística .
GCM (Galois/Counter Mode)	É um modo de operação que fornece simultaneamente confidencialidade e autenticidade (esquema AEAD - <i>Authenticated Encryption with Associated Data</i>). A autenticidade é garantida através de uma etiqueta de autenticação (tag) gerada durante a cifra e verificada antes da decifra. Dispensa o uso de um esquema MAC separado.

1.2. Padding

Como a cifra opera em blocos de tamanho fixo, é necessário completar o último bloco da mensagem se este não estiver completo. O *padding* deve ser **inequívoco** para ser removido corretamente na decifra. Uma solução ingênua como preencher com zeros é **problemática** se a mensagem original terminar naturalmente com zeros, pois o sistema não saberá diferenciar o *padding* dos dados reais. O esquema **PKCS#7** é o mais comum: preenche com N bytes todos com o valor N .

1.3. Características Gerais

- **Determinismo:** A cifra em bloco pura (ex: $AES(k)(b)$) é determinística. Modos de operação com IV (CBC, GCM) tornam o processo não-determinístico.
- **Dimensão da Chave:** Em comparação com a criptografia assimétrica, as **chaves simétricas são mais curtas** (ex: 128, 256 bits) para um nível de segurança equivalente (ex: AES-256 vs. RSA-3072).
- **Reutilização de Chaves:** As chaves simétricas são tipicamente **reutilizadas por longos períodos (meses ou anos)**, mas é crítico que o par (chave, IV) **nunca se repita** nos modos GCM ou CBC para evitar quebras catastróficas de segurança.

2. Funções de Hash Criptográfico e MAC

2.1. Funções de Hash Criptográfico

Uma função de hash criptográfica, $H(m) = h$, mapeia uma mensagem de tamanho arbitrário m para um resumo h de tamanho fixo. As propriedades essenciais são:

- **Resistência à Pré-imagem:** Dado h , é computacionalmente inviável encontrar m tal que $H(m) = h$.
- **Resistência à Segunda Pré-imagem:** Dado m_1 , é inviável encontrar $m_2 \neq m_1$ tal que $H(m_2) = H(m_1)$.
- **Resistência a Colisões:** É inviável encontrar quaisquer dois $m_1 \neq m_2$ tais que $H(m_1) = H(m_2)$.

Implicações de Segurança:

- Se uma função de hash (como **MD5** ou **SHA-1**) tiver a sua resistência a colisões comprometida, um atacante pode criar dois documentos diferentes com o mesmo hash. Isto é **devastador para assinaturas digitais** e para a emissão de **certificados X.509**, pois permite que um certificado fraudulento tenha a mesma assinatura que um legítimo.

2.2. Código de Autenticação de Mensagem (MAC)

Um **MAC** é uma etiqueta criptográfica $t = MAC(k, m)$ que garante a **integridade** e a **autenticidade da origem** da mensagem m , partindo do princípio de que apenas as entidades que conhecem a chave secreta k a podem gerar e verificar.

- **Distinção fundamental:** Um MAC usa **criptografia simétrica** (chave secreta partilhada). **Não garante não-repúdio**, pois qualquer entidade que saiba gerar o MAC também o sabe verificar.
- **Dimensão do MAC:** A dimensão da marca t **depende do algoritmo utilizado** (ex: HMAC-SHA256 gera uma marca de 256 bits), mas **não depende do tamanho da mensagem m** .
- **HMAC (Hash-based MAC):** É a construção de MAC mais comum, definida como $HMAC(k, m) = H((k' \oplus opad) \parallel H((k' \oplus ipad) \parallel m))$, onde k' é uma chave derivada de k , e $opad$, $ipad$ são constantes.

3. Criptografia Assimétrica e Assinatura Digital

3.1. Criptografia de Chave Pública

Utiliza um **par de chaves matematicamente relacionadas**:

- **Chave Pública (K_{pub}):** Distribuída publicamente, usada para **cifrar** mensagens ou **verificar** assinaturas.
- **Chave Privada (K_{priv}):** Mantida em segredo pelo seu dono, usada para **decifrar** mensagens ou **assinar** documentos.

Característica importante: A cifra assimétrica é computacionalmente muito mais pesada que a simétrica. É comumente usada para **negociar uma chave de sessão simétrica** (ex: no TLS), e não para cifrar os dados da aplicação diretamente.

3.2. Assinatura Digital

É o equivalente digital de uma assinatura manuscrita.

- **Processo:**

1. O remetente gera o **hash** do documento.
2. O remetente **cifra este hash com a sua chave privada (K_{priv})**. O resultado é a **assinatura digital**.
3. O destinatário recebe o documento e a assinatura.
4. O destinatário **decifra a assinatura com a chave pública do remetente (K_{pub})**, obtendo o hash original (**h_1**).
5. O destinatário calcula o **hash** do documento recebido (**h_2**).
6. Se **$h_1 == h_2$** , a assinatura é **válida**.

Propriedades garantidas:

- **Autenticidade da Origem:** A assinatura prova que o documento veio do dono de K_{priv} .
- **Integridade:** Qualquer alteração no documento invalidará o hash.
- **Não-Repúdio:** O remetente não pode negar ter assinado o documento, pois apenas ele possui a chave privada. Esta propriedade **não é garantida por um MAC**.

Por que não usar um MAC em vez de uma assinatura digital? Um MAC requer uma **chave secreta partilhada** entre o assinante e o verificador. Para um sistema público de verificação (ex: verificar a assinatura de um contrato ou de um certificado), partilhar a chave secreta com todos os verificadores anularia o não-repúdio e seria um pesadelo de gestão de chaves.

4. Infraestrutura de Chave Pública (PKI) e Certificados X.509

4.1. O Problema da Distribuição de Chaves Públicas

Como pode uma entidade confiar que uma dada chave pública pertence realmente a uma identidade específica (ex: www.exemplo.com)? A solução é a PKI.

4.2. Certificados X.509

Um certificado digital é um documento eletrónico que **vincula uma chave pública a uma identidade** (o *Subject*). Este vínculo é atestado pela **assinatura digital** de uma entidade confiável chamada **Autoridade de Certificação (CA)**.

Conceitos-chave:

- **Cadeia de Confiança:** Uma hierarquia de certificados.
 1. **CA Raiz (Root CA):** Autoassinada. É a **âncora de confiança**. A sua chave pública deve estar instalada de forma segura no sistema operativo ou navegador (*TrustStore*).
 2. **CA Intermédia (Intermediate CA):** O seu certificado é assinado pela CA Raiz (ou por outra CA Intermédia).
 3. **Certificado Folha (End-entity Certificate):** Pertence à entidade final (ex: servidor web www.isel.pt). É assinado por uma CA Intermédia ou diretamente pela Raiz.
- **Verificação da Assinatura de um Certificado:** A assinatura de um certificado folha é **verificada usando a chave pública do seu emissor** (a CA Intermédia), que por sua vez está contida no certificado dessa CA Intermédia. **A chave pública dentro do certificado folha serve apenas para ser usada pela entidade final, não para verificar a assinatura do seu próprio certificado.**
- **Certificado Autoassinado:** É um certificado onde o *Subject* e o *Issuer* são o mesmo. É usado para **CAs Raiz** ou para testes. **Não é universalmente considerado de confiança** a menos que explicitamente

adicionado a uma *TrustStore*.

4.3. Revogação

Mecanismos como **CRL (Certificate Revocation List)** ou **OCSP (Online Certificate Status Protocol)** são usados para verificar se um certificado foi revogado pela sua CA antes do seu prazo de validade expirar.

[VERIFICAR: O tema OCSP/CRL não é explicitamente abordado nos exames, mas é parte integrante da PKI.]

5. Java Cryptography Architecture (JCA)

A JCA é uma *framework* que fornece uma arquitetura para acesso a funcionalidades criptográficas em Java.

5.1. Princípios Fundamentais

- **Independência de Algoritmo:** A API é genérica. Para usar AES ou DES, o código é o mesmo, mudando apenas o nome do algoritmo passado ao método fábrica `getInstance()`. Exemplo:
`Cipher.getInstance("AES/CBC/PKCS5Padding")`.
- **Independência de Implementação:** A JCA permite que múltiplos *providers* criptográficos (ex: SunJCE, Bouncy Castle) forneçam implementações concretas dos algoritmos, sendo um deles selecionado em tempo de execução.

5.2. Classes Principais e Utilização

- **Cipher (Motor de Cifra):** Fornece funcionalidades de cifra/decifra.
 - **Métodos `update()` e `doFinal()`:** A separação existe porque a **Cipher** pode processar dados em **partes** (*streaming*). `update()` processa blocos de dados e pode devolver parte do criptograma. `doFinal()` processa o último bloco (executando o *padding*, se necessário) e finaliza a operação.
 - **Modos de Operação da **Cipher** (`init()`):**
 - **ENCRYPT_MODE / DECRYPT_MODE:** Para cifrar/decifrar dados gerais.
 - **WRAP_MODE / UNWRAP_MODE:** Para **cifrar/decifrar chaves criptográficas**. É uma forma segura de transportar ou armazenar material de chave, usando outra chave (ex: uma chave simétrica de transporte ou uma chave pública RSA).
- **Mac (Motor de MAC):** Usado para gerar e verificar Códigos de Autenticação de Mensagem. É inicializado com uma **SecretKey**.
- **X509Certificate:** Representa um certificado X.509. Métodos relevantes:
 - `getPublicKey()`: **Retorna a chave pública do certificado** (pertencente ao *Subject*).
 - `checkValidity()`: Verifica a data de validade do certificado.
 - `verify(PublicKey issuerKey)`: **Verifica a assinatura do certificado** usando a chave pública do emissor.

6. Protocolo Transport Layer Security (TLS)

O TLS opera sobre o TCP e fornece um canal de comunicação seguro entre duas partes, garantindo **confidencialidade, integridade e autenticação**.

6.1. Sub-protocolos

- **Handshake Protocol:** Negocia os parâmetros da sessão segura (versão do protocolo, *cipher suites*), autentica o servidor (e opcionalmente o cliente) e estabelece as **chaves de sessão**.
- **Record Protocol:** Usa as chaves de sessão estabelecidas pelo *handshake* para **cifrar e autenticar todas as mensagens de dados da aplicação** (ex: pedidos e respostas HTTP). É aqui que a confidencialidade e integridade dos dados de aplicação (como *cookies*) são efetivamente garantidas.

6.2. Processo de *Handshake* e Segurança

- **Negociação e Ataques de Downgrade:** Cliente e servidor trocam mensagens *ClientHello* e *ServerHello* onde acordam a versão do TLS e a *cipher suite*. Um atacante pode tentar modificar estas mensagens para forçar o uso de protocolos antigos e inseguros (ex: SSLv3). Este ataque é **detetado nas mensagens finais do handshake (*Finished*)**, que contêm um **MAC calculado sobre todas as mensagens de handshake trocadas**. Se um atacante modificar qualquer mensagem anterior, o MAC no *Finished* será inválido e a ligação será terminada.
- **client_random e server_random:** Estes valores aleatórios, trocados em claro durante o *Hello*, são essenciais para a **derivação das chaves de sessão**. Juntamente com o *pre-master secret*, garantem que as chaves de cada sessão são únicas, mesmo que o *pre-master secret* seja o mesmo (ex: se o RSA estático for usado). A sua modificação também é detetada pelo *Finished*.
- **Perfect Forward Secrecy (PFS):** É uma propriedade que garante que a **compromissão da chave privada de longa duração do servidor não permite decifrar sessões TLS passadas**. Isto é conseguido através de *cipher suites* baseadas em acordo de chaves efémeras, como **DHE (Diffie-Hellman Ephemeral)** ou **ECDHE**. Se a chave privada RSA do servidor for comprometida, sessões passadas que usaram RSA para cifrar o *pre-master secret* podem ser decifradas, ao contrário das que usaram ECDHE. O PFS resolve ataques à chave de longa duração do servidor (parte ii) e ao armazenamento de sessões passadas no cliente/servidor.

7. Autenticação de Utilizadores e Armazenamento de *Passwords*

7.1. Más Práticas e Implicações

- **Armazenar passwords com cifra reversível ($E(k)(\text{pwd_u})$):** Se a chave *k* for armazenada na mesma base de dados que as *passwords* cifradas (*p_u*), um atacante com uma cópia da BD obtém **acesso imediato a todas as passwords em claro**, bastando decifrar *p_u* com *k*. Isto é catastrófico.

7.2. Boas Práticas para Armazenamento Seguro

O objetivo é armazenar um "verificador" (*v_u*) que permita confirmar uma *password*, mas que seja inútil para um atacante que roube a base de dados.

- **Função de Derivação de Chave Baseada em Password (ex: PBKDF2, bcrypt, scrypt, Argon2):** Estas funções são computacionalmente **lentas** e usam um **salt**.
- **salt (*s_u*):** Um valor aleatório **único por utilizador**. É armazenado em claro juntamente com o verificador.
 - **Objetivo do salt:**
 1. **Mitigar ataques de dicionário pré-computados (*Rainbow Tables*):** Ataques pré-computados tornam-se inviáveis porque cada *password* é combinada com um *salt* diferente, exigindo uma pré-computação única para cada (*password*, *salt*).

2. **Mitigar ataques a múltiplos utilizadores em simultâneo:** Duas *passwords* iguais terão verificadores diferentes se os *salts* forem diferentes, não revelando a igualdade das *passwords*.

- **Construção Recomendada:** $v_u = \text{PBKDF2}(\text{HMAC-SHA256}, \text{password}_u, s_u, \text{iterations})$. A função é executada no lado do servidor a cada tentativa de *login*.
- **Aumentar o tamanho do salt:** Aumentar o *salt* de 16 para 64 bits **não aumenta significativamente a dificuldade de um ataque online à interface de autenticação** (que é limitado pela rede e pelo sistema), mas **aumenta a segurança contra ataques offline à base de dados**, tornando a pré-computação ainda mais impraticável. A afirmação de que aumenta "4 vezes" é enganosa; a complexidade de um ataque de pré-computação aumenta exponencialmente com o tamanho do *salt* (de 2^{16} para 2^{64} tabelas *rainbow* possíveis). [VERIFICAR: O impacto exato depende do modelo de atacante, mas o princípio é que o espaço de pré-computação cresce exponencialmente com o tamanho do *salt*.]

8. Modelos de Controlo de Acessos (RBAC)

O controlo de acessos é o processo de garantir que os utilizadores apenas executam ações para as quais têm autorização. É feito **após a autenticação**.

8.1. Modelo RBAC (Role-Based Access Control)

O RBAC simplifica a gestão de permissões ao introduzir o conceito de **Papel (Role)**. As permissões são atribuídas a papéis, e os papéis são atribuídos a utilizadores.

- **Componentes do RBAC0:**
 - **U:** Conjunto de **Utilizadores**.
 - **R:** Conjunto de **Papéis**.
 - **P:** Conjunto de **Permissões** (ex: *ler_ficheiro*, *escrever_BD*).
 - $UA \subseteq U \times R$: Relação **User-Assignment** (Que utilizadores têm que papéis).
 - $PA \subseteq P \times R$: Relação **Permission-Assignment** (Que permissões estão associadas a que papéis). [Nota: O texto refere *PA* como um conjunto de pares (*role*, *permissão*), por vezes a notação é $P \times R$ ou $R \times P$. O conceito é o mesmo.]
 - **Sessão (S):** Um utilizador autentica-se e estabelece uma **sessão**. Nessa sessão, o utilizador ativa um **subconjunto dos seus papéis** ($roles(s) \subseteq \{ r \mid (u, r) \in UA \}$). Isto implementa o **Princípio do Menor Privilégio**, permitindo que um utilizador (ex: administrador) use apenas os seus privilégios de "utilizador normal" na maior parte do tempo.

8.2. Modelo RBAC1 (Hierarquia de Papéis)

Introduz uma relação de ordem parcial entre papéis, $RH \subseteq R \times R$ (frequentemente denotada por $\prec$). Se $r1 \prec r2$, então $r2$ é **hierarquicamente superior** a $r1$ e **herda todas as permissões de $r1$** . A hierarquia simplifica a administração (ex: o papel "Médico" pode herdar todas as permissões do papel "Funcionário").

- **Nota importante:** *PA* **não relaciona utilizadores diretamente**. A hierarquia (*RH*) é que permite a herança de permissões entre papéis.

8.3. Access Control List (ACL) vs. RBAC

- **ACL:** Lista associada a cada objeto (ficheiro, recurso) que especifica quais os utilizadores e os seus direitos sobre esse objeto. **Vantagem:** É fácil saber quem tem acesso a um objeto específico.
Desvantagem: A gestão de direitos por utilizador (ex: "Quais são todas as permissões do utilizador X?") é complexa, pois exige percorrer as ACLs de todos os objetos.

9. OAuth 2.0 e OpenID Connect (OIDC)

9.1. OAuth 2.0 (Framework de Autorização)

É um protocolo de **autorização delegada**. Permite que uma **Aplicação Cliente** aceda a recursos protegidos de um **Dono de Recursos** (Resource Owner), alojados num **Servidor de Recursos**, sem que o dono partilhe as suas credenciais com a aplicação. Em vez disso, o dono concede **acesso delegado** através de um **Servidor de Autorização**.

Fluxo Authorization Code Grant (o mais comum e seguro):

1. **Pedido de Autorização:** A Aplicação Cliente redireciona o *browser* do Dono de Recursos para o **Authorization Endpoint** do Servidor de Autorização, incluindo **client_id**, **redirect_uri** (URL de *callback* da aplicação) e o **scope** (permissões solicitadas).
2. **Concessão:** O Dono de Recursos autentica-se e autoriza o **scope**.
3. **Código de Autorização:** O Servidor de Autorização redireciona o *browser* de volta para o **redirect_uri** da Aplicação Cliente, incluindo um **authorization_code** (opaco e de uso único).
4. **Troca por access_token:** A Aplicação Cliente faz um pedido direto (servidor-a-servidor) ao **Token Endpoint** do Servidor de Autorização, autenticando-se com o seu **client_id** e **client_secret**, e troca o **authorization_code** por um **access_token**.
5. **Acesso ao Recurso:** A Aplicação Cliente usa o **access_token** para fazer pedidos autenticados ao Servidor de Recursos.

Componentes e Parâmetros-Chave:

- **scope:** Lista de permissões específicas que a aplicação está a solicitar. Definido pelo **cliente**, mas requer aprovação do **dono de recursos**. Ex: **scope=openid profile email**.
- **Parâmetro state:** Valor aleatório gerado e armazenado pela Aplicação Cliente (ex: na sessão do utilizador) antes do redirecionamento inicial. É devolvido pelo Servidor de Autorização na resposta. A aplicação **DEVE** validar que o **state** recebido é igual ao armazenado. É uma **defesa crítica contra ataques de Cross-Site Request Forgery (CSRF)**, impedindo que um atacante injete um código de autorização malicioso na sessão da vítima.

9.2. OpenID Connect (OIDC) (Camada de Autenticação sobre OAuth 2.0)

O OAuth 2.0 é apenas para **autorização** ("O que a aplicação pode fazer?"). O OIDC estende o OAuth 2.0 para fornecer **autenticação federada** ("Quem é o utilizador?").

- **id_token:** É um **JSON Web Token (JWT)** assinado pelo Servidor de Autorização (Fornecedor de Identidade - IdP). Contém **afirmações (claims)** sobre a identidade do utilizador autenticado (ex: **sub**, **email**, **name**). A Aplicação Cliente **valida a assinatura e o emissor do id_token** para confirmar a identidade do utilizador.
- **access_token vs. id_token:**

- **access_token**: É um "cartão de crédito" usado para aceder a APIs protegidas (Servidores de Recursos). O seu conteúdo é **opaco** para o cliente.
 - **id_token**: É um "bilhete de identidade" sobre o utilizador. Destina-se **exclusivamente à Aplicação Cliente**.
 - **UserInfo Endpoint**: Uma API OAuth 2.0 protegida que retorna mais *claims* sobre o utilizador autenticado. A Aplicação Cliente acede a este *endpoint* **apresentando o access_token** obtido com o **scope=openid** (ou scopes adicionais como **profile, email**).
 - **Vulnerabilidade de Mistura de Contas**: Se uma aplicação não validar corretamente a associação entre o estado da sessão local e o **state/id_token** recebido, um atacante pode manipular o fluxo para forçar uma vítima a autenticar-se como o atacante.
-

Lista de Termos-Chave

- **AEAD**: *Authenticated Encryption with Associated Data*
- **AES**: *Advanced Encryption Standard*
- **Assinatura Digital**: *Digital Signature*
- **CA**: Autoridade de Certificação (*Certification Authority*)
- **CBC**: *Cipher Block Chaining*
- **Certificado X.509**: *X.509 Certificate*
- **Cipher Suite**: Conjunto de algoritmos criptográficos negociados no TLS
- **CSRF**: *Cross-Site Request Forgery*
- **ECB**: *Electronic Code Book*
- **GCM**: *Galois/Counter Mode*
- **HMAC**: *Hash-based Message Authentication Code*
- **IV**: Vetor de Inicialização (*Initialization Vector*)
- **JCA**: *Java Cryptography Architecture*
- **JWT**: *JSON Web Token*
- **MAC**: *Message Authentication Code*
- **Não-Repúdio**: *Non-Repudiation*
- **OAuth 2.0**: *Open Authorization 2.0*
- **OIDC**: *OpenID Connect*
- **Padding**: Preenchimento de bloco (ex: PKCS#7)
- **PFS**: *Perfect Forward Secrecy*
- **PKI**: Infraestrutura de Chave Pública (*Public Key Infrastructure*)
- **RBAC**: *Role-Based Access Control*
- **Salt**: Valor aleatório usado no armazenamento de *passwords*
- **Scope**: Parâmetro de permissões em OAuth 2.0 / OIDC
- **TLS**: *Transport Layer Security*
- **TrustStore**: Repositório de certificados de confiança (CAs raiz)
- **UA / PA**: *User-Assignment / Permission-Assignment* (em RBAC)